

0__Overview, Workflow, and Identification

Overview, Workflow, and Identification

Version 2, 2026-09-04

This document is the complete working standard for a trainer turning a mined package into a deliverable non-connector Harbor task. Work through the stages in order. Every point carries an identifier you can quote: stages are numbered (Stage 7), steps within a stage are numbered (7.3), points within a step are lettered (7.3.b), and rules from the Rule Inventory keep their inventory identifiers (VER-8). The Reviewer Document uses the same identifiers and adds no requirement of its own.

Part 0. How to use this document

0.1 What this document is. The stages below are the order in which a task is built: receive the mined package, fix the boundary of what the agent may see, design the difficulty, build the fixture, write the instruction, derive the golden answer, write the verifier, decide on any judge-based check, build the harness, set the environment and manifest, prove the task to yourself, run it, assemble the evidence, and write the package files. Each stage says what to do, what to take special care over, and which rules apply. Each stage ends with a checklist of rule identifiers. The full text of every rule, with its meaning, the failure it prevents, a bad and a good example, the hand check and the tool that detects it, is in Appendix A. You should be able to complete a task using this document alone.

0.2 Identifiers. Quote the stage-step-point identifier for anything in the workflow (for example 7.4.c), the inventory identifier for any rule (for example INS-2), and the pattern identifiers for the fair-verifier material (O1 to O17, E1 to E14, B1 to B8) and the stumping tips (T1 to T15). Every identifier in this document is unique.

0.3 Severity.

- **Illegal:** the verifier grades something the disclosed set never established. A correct agent cannot win. The package is returned automatically.
- **Blocking:** the package is returned with both fix branches named.
- **Warning:** noted; fixed unless you record a reason to keep it.
- **Policy:** a recorded decision rather than a check.

0.4 By hand, and by tool. Every rule states a hand check and a tool. Tool names are **client:** followed by the script name for the client's own scripts, **qc:** followed by the check name for the

QC tool built from this standard, and `manual` where only judgement applies. Check everything by hand before submission. The tool lags this document: a PASS from the tool discharges nothing the tool does not implement, and where they disagree this document governs.

0.5 The example task. Every worked example in this document comes from one task, built and run under this standard. The context: Ashcombe Fabrication Ltd, a manufacturer, is closing its quarter to 31 August 2026. The accounts-payable lead asks for every supplier invoice received between 1 June and 31 August to be classified as `compliant`, `exception_applied` (overdue but covered by an approved exception) or `overdue`, as at 31 August. The agent gets six input files: the invoice register, the purchase ledger, the full supplier directory, an exception register, a statement of account one supplier sent, and the contract schedule that states the rules. The agent leaves three files: a CSV table with one row per invoice and its total, currency and finding; a JSON summary of four counts; and a memo giving, for every flagged invoice, its receipt date and the clause or exception that decides it. The package's README records every measurement quoted here. It references no real delivered task.

0.6 Open questions. `OPEN-QUESTIONS.md` lists the policy questions still with the team or the client. A rule that depends on one carries its number, for example [Q12]. Until an item is settled, follow the rule as written.

0.7 How the client audits a package. Five layers. Layer 1: the package defines one coherent, realistic, gradable task (consistency, clarity, realism, leakage). Layer 2: the checked-in evidence, four GLM-5.2 difficulty runs, one non-oracle passing run, and repeat grading for stability. Layer 3: the oracle scores exactly 1.0 on two providers. Layer 4: stronger models and their harnesses run the task. Layer 5: qualitative review of every trial, above all verifier fairness. Some subchecks are mandatory and cannot be averaged away; the two that return most packages are `instruction_verifier_consistency` (the verifier enforces something the instruction never asked for) and `failure_cause_validity` (a failure was counted as difficulty when it was a defect). Every rule in Appendix A names the subcheck it protects, and every checklist line below repeats it.

0.8 Strict pass. A trial passes strictly when its reward is exactly 1.0. The difficulty band, the solvability run and the oracle are all judged on strict passes. There is no partial credit that protects a task from a single unfair check: one check that flips, one cosmetic check that fails on a correct answer, one boundary the key resolves both ways, and a correct run fails.

0.9 The two ways a task is rejected. Either the verifier is unfair (it grades a fact the agent could not know, a form the instruction never fixed, or a proxy for the requirement), or the difficulty is fake (the task is hard only because a rule was left unstated, a boundary left ambiguous, a tolerance set hair-thin, or the population padded). Nearly every rule prevents one

of the two.

0.10 What you submit. The client zips one folder, whose name is the task name. Nothing else at the top level; nothing loose inside `evaluations/`. Under `tests/` you may add the harness scripts, the fixture generator and the discrimination harness; nothing else may be added anywhere.

None

- `<task-folder>/`
- `task.toml`
- `instruction.md`
- `review.csv` one row per client review area
(14.3)
- `qc_report.html` the QC tool's report at the
submitted checksum
- `README.md` the change summary (14.2)
- `environment/`
- `Dockerfile`
- `input/` everything the agent may see
besides the instruction
- `solution/`
- `solve.sh`
- `files/` the golden deliverables
- `golden_trajectory.json` a model's reward-1.0 run, never
the oracle
- `tests/`
- `verifier.json`
- `test_outputs.py`
- `test.sh`
- `rl_world_verifiers/` the vendored engine, unmodified
- `evaluations/`
- `solvability/r1/` one reward-1.0 run, any model,
never the oracle
- `difficulty/r1/ .. r4/` four GLM-5.2 runs
- `oracle/rN/` added by Turing's team after
submission
- `stability/rN/` added by Turing's team after
submission

Each run folder under `evaluations/` holds `agent/trajectory.json`, `result.json`, `config.json`, `verifier/reward.json`, `verifier/reward.txt`, `verifier/ctrf.json`, `verifier/test-stdout.txt`, `artifacts/app/` with every deliverable exactly as graded, and `artifacts/manifest.json`; difficulty runs also hold `verifier/verifier_summary.json`. Stage 13 says what each must contain; Stage 14 says what the README and `review.csv` must say.

Appendix D. Identifier index

- Stages 1 to 15 and their steps and points: this document.
- DIS, INS, FIX, GLD, DIF, VER, LLM, HAR, EXP, ENV, TOML, TRL, EVD, SEC, PKG: the Rule Inventory, reproduced in Appendix A. RVW rules are in the Reviewer Document.
- O1 to O17: the verifier patterns (7.14). E1 to E14: the equivalence suite (11.1). B1 to B8: the breaking suite (11.2). T1 to T15: the stumping tips (3.4). Q1 to Q13: `OPEN-QUESTIONS`.

Counts. 205 identifiers, of which 202 are active rules: INS 17 · DIS 7 · FIX 15 · GLD 11 · VER 30 · LLM 15 · HAR 11 · ENV 10 · TOML 13 · DIF 10 · EXP 12 · EVD 14 · TRL 8 · SEC 6 · RVW 9 · PKG 14. EXP-9 is merged into VER-25; FIX-1 and EVD-5 are withdrawn (2026-09-04) and keep their identifiers so nothing rennumbers.

1_2__Initial Setup and Disclosed Boundaries

Initial Setup and Disclosed Boundaries

Stage 1. Receive the mined package and understand it

1.1 What you receive. The mining team hands you five components and nothing else:

- `task.toml`, usually with an empty `artifacts` list and no judge section.
- `instruction.md`, short, with a template block at the end.
- `environment/Dockerfile` and `environment/input/` with a small fixture.
- `solution/solve.sh` and `solution/files/` with the golden deliverables.
- `tests/verifier.json`, `tests/test_outputs.py`, `tests/test.sh`, and the vendored engine.

There is no README, no `review.csv`, no QC report, no trajectory and no `evaluations/`. Nothing in the package is fixed (Rule PKG-2, ruling R17).

1.2 Inventory the files. Before reading anything, list every file with its size and confirm the five components are all present. Record the list; it is the "mined version" your README will describe departures from (PKG-2).

1.3 Read the instruction and every input, then write your understanding. In one paragraph, in your own words: who is asking, what they need, what the deliverables are, which input carries which fact, and what rules decide the answer. Keep the paragraph; it becomes the README's opening.

1.4 Measure the current complexity. Do all of the following before changing anything:

- **1.4.a** Run the mined verifier against the mined golden files. Note the reward and the number of checks.
- **1.4.b** Count the fixture: rows, columns, files, distinct values. A fixture with one row per audited unit and a status column is a lookup, not an audit (FIX-3).
- **1.4.c** For each finding in the golden table, ask which rule decided it and how many sources you had to open. If every finding comes from one file and one rule, there is no difficulty yet (DIF-2).
- **1.4.d** Read `verifier.json` end to end. Count how many checks are existence checks, keyword checks on prose, or boilerplate.

1.5 Expect the template defects. The mining template ships with defects you must remove and list as departures (PKG-14). Check the list rather than assuming it.

1.6 Decide what to keep. Keep the domain and the deliverable idea if they are realistic. Keep the engine untouched (VER-30). Expect to replace the fixture, the instruction's template block, most of the verifier and the harness.

1.7 Start the departures log now. Open a scratch file and record every change from this point on. The README's departures section (PKG-2) is written from it.

Stage 2. Fix the boundary of what the agent can see

2.1 The disclosed set. The agent sees `instruction.md` and every file under `environment/input/`, because those are the only things the Dockerfile copies (ENV-1). Nothing else exists for it. Every expectation in the verifier must be derivable from the disclosed set (DIS-1).

2.2 Run the blind-reader test on the mined package (DIS-6). Give the instruction and the inputs to someone who has not seen the verifier. Ask them to list every literal string, number and identifier the deliverables must contain. Compare with the literals in `verifier.json`. Anything the verifier knows and the reader does not is illegal.

2.3 Decide the deliverables and their carriers (PKG-13). A workbook where a practitioner would file a workbook; a document where they would circulate a document; CSV or JSON for machine summaries. Each carrier decides what the engine can grade (7.4).

2.4 Decide what is stated and what is assumed (DIS-3, DIS-4). Field-standard knowledge need not be restated. Anything dataset-specific or arbitrary must be disclosed. Where you leave a representation choice open (case, whitespace, order), the verifier must accept every reasonable form.

2.5 Special attention.

- **2.5.a** Nothing about process is graded: not tool choice, not narration, not "opened every file" (DIS-5).
- **2.5.b** The undisclosed set never enters the image: no `COPY . /app`, no `solution/`, no `tests/`, no README (DIS-7, ENV-1).
- **2.5.c** A count of the population, a name of the outlier, a field name that reveals the deciding mechanism: each is a leaked answer (INS-8, INS-9).

A.1 DIS: the disclosure boundary

- **DIS-1:** The verifier enforces only what is derivable from the disclosed set. (Illegal)
- **DIS-2:** Every check's `why_justification` cites a sentence in the disclosed set.

(Illegal)

- **DIS-3:** Standard conventions may be assumed; arbitrary choices must be stated.

(Blocking)

- **DIS-4:** Unstated representation rules are tolerated, never enforced. (Blocking)
- **DIS-5:** Nothing about process, tool choice, narration, or trajectory is graded. (Illegal)
- **DIS-6:** The blind-reader test is run before submission. (Blocking)
- **DIS-7:** The undisclosed set never enters the image. (Blocking)

3__Difficulty and Coverage Design

Difficulty and Coverage Design

Stage 3. Design the difficulty

3.1 Core Principle. A well-designed task forces an AI agent to do most of its work correctly, only to fail on one clear, decisive point. The fully disclosed information must yield exactly one correct answer—one that a diligent reader finds, but a hasty reader misses. To test your task's difficulty, apply the **Disclosure Rule (DIF-1)**: write every decisive rule as a single sentence and review the list. If the task suddenly becomes trivial, your difficulty came from hiding information rather than genuine reasoning complexity.

3.2 Model Behavior & Why Sanctioned Patterns Work. Standard language models (like GLM-5.2) tend to stop working at the first plausible success ("green result"). Typically, the model reads the most obvious file, applies a default rule, checks its initial results against the surface-level data, and submits its work. Sanctioned patterns intentionally place the critical deciding factor where a shallow check won't see it, while ensuring the correct answer remains 100% determinable from the provided materials.

3.3 Overview of Sanctioned Patterns. Every pattern listed below leads to a single, unambiguous correct answer derivable directly from the disclosed documentation (DIF-3). They test deep comprehension and edge-case validation rather than obscure or missing facts.

Sanctioned Patterns Summary (3.3)

Pattern	The mechanism	In the example
Unit-of-analysis trap	the row's key is one level below the unit the rules govern	the ledger has line items; terms, exceptions and totals apply per invoice
Stale authority	use the revision in force at the relevant date, not the latest	one exception revised after the extract date; one narrowed by a revision that is in force

Wrong-default lure	the obvious heuristic is almost right and wrong on a few records	terms run from receipt, not the supplier's issue date; eight invoices change finding
Latent crux	a real rule that never fires on the visible sample	the non-sterling allowance changes the finding on one of the five foreign-currency rows, and its "greater of" wording on one more
Misdirection with a disclosed tie-breaker	a source that disagrees, and a sentence saying which governs	the supplier statement calls an invoice settled; the request says the ledger governs
Entangled rules	one fact changes two rules at once	not used in the example
Ordering assumption	the data is not in the order the reader assumes	the ledger is exported grouped by supplier, not by date

3.4 Key Strategies for Creating Challenging Tasks. Use these actionable guidelines to test the model's true reasoning capabilities effectively:

- **T1 Hide the key case:** Position the deciding data point outside the primary or default dataset slice.
- **T2 Craft realistic lures:** Make common shortcut heuristics yield almost-correct answers that fail on key records.
- **T3 Supply conflicting sources:** Include an authoritative-looking incorrect source, but clearly disclose which document takes precedence.
- **T4 Define explicit timeframes:** Ensure all data lookups rely on specific, time-bound parameters.
- **T5 Challenge ordering assumptions:** Present data organized differently from how readers naturally expect (e.g., grouped by entity rather than sorted by date).

- **T6 Document rule interactions:** Combine multiple rules so they influence each other, and explicitly write down how they interact.
- **T7 Align rules above record level:** Set the primary evaluation rules one level higher than individual data rows, and document this scope in governing instructions.
- **T8 Require relational joins:** Force the reader or model to cross-reference multiple datasets to determine the correct result.
- **T9 Use uniform sample data:** Keep visible sample rows identical along key variables to prevent simple pattern-matching.
- **T10 Create rule collisions:** Build overlapping rule precedence scenarios that require careful analysis (DIF-5).
- **T11 Aim for clear failures:** Focus on failures caused by fundamental logic oversights rather than minor formatting errors (DIF-6, DIF-9).
- **T12 Pre-define incorrect paths:** Document expected wrong interpretations in the README before running test trials (DIF-4).
- **T13 Verify full disclosure:** Continuously test that all necessary rules are stated clearly without relying on unwritten assumptions.
- **T14 Monitor performance trends:** Evaluate model convergence and pass/fail consistency across test runs (TRL-3, TRL-4).
- **T15 Avoid artificial difficulty:** Do not create difficulty using hidden rules, subjective grading, formatting tricks, ambiguous boundaries, tight numerical tolerances, extreme document volume, misleading phrasing, or arbitrary timeouts (DIF-6, EXP-12).

3.5 What Does Not Count as Difficulty. Artificial barriers listed in T15 lead to task defects rather than valid evaluation. In particular, sheer data volume does not equal genuine reasoning difficulty (FIX-8).

3.6 Target Pass Rate & Validation Band. A task is properly calibrated if GLM-5.2 strictly passes at most 3 out of 4 test runs. Passing 4 out of 4 runs means the task is too easy and will be rejected (DIF-7). Only valid logic failures count toward this score. Stronger models will also evaluate tasks post-delivery to catch tasks that are universally easy across models (DIF-10).

3.7 Documenting Your Design. Record all selected patterns, deciding rules, and anticipated incorrect readings directly into the task's README draft during development.

A.5 DIF: difficulty and coverage

- **DIF-1:** Difficulty survives full disclosure. (Blocking)
- **DIF-2:** Conceptual, not clerical. (Blocking)
- **DIF-3:** The crux is a sanctioned pattern. (Blocking)
- **DIF-4:** Expected wrong readings are declared in the README before trials. (Blocking)

- **DIF-5:** Precedence interactions are exercised. (Blocking)
- **DIF-6:** Volume, ambiguity, and hair-thin tolerances are not difficulty. (Blocking)
- **DIF-7:** Pass band: at most 3 of 4 GLM-5.2 strict passes; 4/4 is rejected. (Blocking)
- **DIF-8:** A stronger model may diagnose a 0/4 task; that score is never reported. (Warning)
- **DIF-9:** Aim for the good-failure signature. (Warning)
- **DIF-10:** Policy: the client's Layer 4 runs three lanes × two attempts [Q7]. (Warning)

3.8 “good difficulty” vs “bad difficulty”

Good difficulty	Bad difficulty
Requires finding the relevant evidence	Requires searching endlessly
Requires applying disclosed rule	Requires guessing undisclosed rule
Has one determinate answer	Has multiple defensible answers
Exploits realistic reasoning shortcut	Relies on trick wording
Requires meaningful cross-reference	Requires unnecessary document volume
Failure is categorical	Failure depends on tiny tolerance
Difficulty survives disclosure	Difficulty disappears when rules are disclosed

4__Fixture Building and Golden Answers

Fixture Building and Golden Answers

Stage 4. Build the fixture

4.1 State the facts a reader needs to use the data (FIX-2, FIX-10, FIX-14, INS-10, INS-16).

The instruction, or the governing document it names, states: what each input file is, in a sentence; the unit of audit; the period and the as-of date; units, encoding and date conventions; the meaning of any column that is not obvious from its name; and which source governs when two disagree. Nothing about the data may be left for the agent to guess.

4.2 Write the governing document (FIX-2, INS-2, INS-3, FIX-14). The contract, policy or specification the findings must follow. Every threshold says in words which side an exact hit falls on. Every tie-break and precedence is stated and reflects real practice. Every temporal rule has an as-of date. The unit of audit is stated.

4.3 Do not hand the population over (FIX-3). No input carries a counterpart to any graded figure. No file has one row per audited unit when the unit itself must be worked out. Directories are larger than the population.

4.4 Make distractors derivable from fields (FIX-4). A superseded exception is superseded because a later revision is effective by the relevant as-of date, never because its identifier ends in `-OLD` or it is stored in a file called `archived`.

4.5 Put a record exactly on every threshold (FIX-5). For each numeric threshold, one record sits exactly at the boundary and the governing document says which side it falls on.

4.6 Give every rule a record it alone decides, and every source a distractor (FIX-6, FIX-7). For each rule, some record's finding changes if and only if that rule is applied. Each input file contains something that looks applicable and is not.

4.7 Cut redundancy (FIX-8). Group records by (finding, deciding rule, exception state, boundary membership). Many records in one group is volume. Each record should add a signature or exercise a collision.

4.8 Generate, do not hand-edit (FIX-9). A seeded script produces the inputs and the key together, lives under `tests/`, and is never copied into the image. Regenerating after an edit is one command.

4.9 Conventions (FIX-10). Text inputs use UTF-8, LF and no BOM. Tabular inputs have a header row. Date conventions e.g. ISO dates are consistent by file type and are stated once in

the instruction or governing document.

4.10 Hashes (FIX-11). Record each input's SHA-256 under `tests/`. Every verifier expectation is fixed from these pinned inputs at build time; nothing on the grading path reads the live inputs.

4.11 Identifiers, dates and size (FIX-12, FIX-13, FIX-15, FIX-16). Short human identifiers (`INV-0417`, `EX-2901`). No date after the extract date unless the instruction or the governing document says such dates exist. Plausible values with cents and a realistic spread. A size the agent can read within the budget.

4.12 Special attention.

- **4.12.a** Identifiers must not follow the scenario order.
- **4.12.b** The instruction and the governing document describe the *fields* the traps live in without describing the traps.
- **4.12.c** A statement or summary that disagrees with the governing source must be realistic in the source's own format.

A.3 FIX: input fixtures

- **FIX-2:** The unit of audit is stated in a governing document. (Blocking)
- **FIX-3:** No input carries a counterpart to any graded figure. (Illegal)
- **FIX-4:** Distractor status is derivable from fields, never from a name or label. (Blocking)
- **FIX-5:** Every threshold has at least one record exactly on it. (Blocking)
- **FIX-6:** Every rule has at least one record where it alone decides the outcome. (Blocking)
- **FIX-7:** Every evidence source carries at least one distractor that must be actively rejected. (Blocking)
- **FIX-8:** Every record adds a distinct rule signature; redundancy is cut. (Warning)
- **FIX-9:** Fixtures come from a seeded generator kept out of `input/`, shipped with the package. (Blocking)
- **FIX-10:** Text inputs use UTF-8, LF and no BOM; tabular inputs have a header row; date conventions are consistent and stated once. (Blocking)
- **FIX-11:** Inputs are read-only in the image and hash-pinned. (Blocking)
- **FIX-12:** Identifiers are short and human; no filename bears an answer. (Blocking)
- **FIX-13:** No date in the fixture falls after the extract date. (Blocking)
- **FIX-14:** Every temporal rule has a stated as-of date. (Blocking)
- **FIX-15:** Fixture values are plausible for the domain. (Blocking)
- **FIX-16:** Fixture size is proportionate to the run budget. (Warning)

5__Instructions and Artifact Requirements

Instructions and Artifact Requirements (Stage 5)

Voice and Tone

- **Natural Framing (INS-7, INS-14):** Write the instruction as a natural request from a colleague with a clear reason. It must sound human (Warning) and never coach the agent (Blocking).
- **Omit Artificial Structure:** Remove headings, roleplay, preambles, step-by-step lists, and tool lists. Never include sentences that name the trap or enumerate reasoning dimensions.
- **Read-Aloud Test:** Read the instruction aloud and strike any sentence a real colleague would not have written.

Required Instruction Content

- **Context & References (INS-10):** Define what each input file is and where it is located using a single sentence per file. Every referenced file, table, or document must exist in the disclosed set (Blocking).
- **Deliverable Paths (INS-15):** Name each deliverable exactly once using absolute ``/app`` paths in backticks. Spelling must perfectly match `task.toml` and `verifier.json` (Blocking).
- **Output Formatting (INS-5, INS-6):** Fully specify formats, including exact filenames, column sets/orders (or "any order"), number formats/precision, date formats, encodings, null/boolean tokens, sort orders, and JSON key sets/types (Blocking). Format completeness is a baseline, never the source of difficulty (Warning).
- **Enum Literals (INS-4):** Provide enum literals verbatim and explicitly state whether case sensitivity matters; fix the case once (Blocking).
- **Prose Deliverables (INS-12):** Explicitly state the required content for prose and clarify whether it will be graded on those facts (Blocking).
- **Conflict Resolution (INS-16):** Specify exactly which document governs when sources disagree (Blocking).
- **Example Values (INS-17):** Mark illustrative values clearly as shape-only examples, ensuring they cannot be valid correct answers (Blocking).
- **Missing Fixture Context:** Explicitly state necessary facts not covered in the governing document, such as periods, as-of dates, units, date conventions, and non-obvious column meanings.

Prohibited Instruction Content

- **Unprovable Requirements (INS-1):** Omit requirements that no resulting file could prove, such as action ordering, effort, tone, "final action", or "confirm each file exists" (Illegal).
- **Leaked Answers (INS-8, INS-9):** Never state a target count, total, or specific entity the deliverable is supposed to derive (Illegal). Do not use output field names that reveal the underlying deciding mechanism (Illegal).
- **Harness Meta-Lines (INS-13):** Do not include test constraints like timeouts, resource limits, or "do not use the internet" (Warning).
- **Template Residue (INS-11):** Remove any leftover boilerplate sentences from other tasks or domains (Blocking).

Special Attention & Edge Cases

- **Numeric Thresholds (INS-2):** State whether numeric thresholds are inclusive or exclusive using words within the same sentence; symbols alone are insufficient (Blocking).
- **Precedence (INS-3):** State every tie-break and precedence order clearly, ensuring it reflects real domain practice rather than artificial logic (Blocking).
- **Representation Rules (DIS-4, DIS-6):** If column order, number format, or casing is fixed in the instructions, the verifier may enforce it. If left open, the verifier must tolerate reasonable variations (Blocking).
- **Blind-Reader Test:** The instruction and governing documents combined must allow a reader to list every single literal the verifier will expect without looking at the verifier code.

6__Golden Answer

Golden Answer Derivation & Verifier Pitfalls (Stage 6)

Because `solve.sh` may install precomputed files, a 1.0 oracle score only proves that the golden files match the verifier's expectations (Warning: GLD-1). Therefore, manual, independent derivation checks are the only reliable controls on key correctness.

Key Validation & Derivation Steps

- **Verify Self-Consistency:** Group the key by governing fields; identical fields must always produce identical outputs to prevent logic defects (Blocking: GLD-2).
- **Confirm Rule Application:** List every record triggered by each rule and ensure the golden key reflects the correct application on all of them (Blocking: GLD-3).
- **Derive Boundaries Manually:** Re-derive every boundary record on paper directly from the governing documents, strictly avoiding the use of code or generators (Blocking: GLD-4).
- **Resolve Ambiguities:** If a threshold or tie-break can be read two ways, re-derive the key under both. If the key changes, you must add a clarifying sentence to the instruction rather than just picking a reading (Blocking: GLD-5).
- **Reconcile Prose Figures:** Recompute every number in the golden memo using the structured deliverables (table/JSON) to ensure perfect alignment (Blocking: GLD-6).
- **Remove Construction Leaks:** Search the golden prose for words like "intended", "should be", "we set", or "resolved by" to ensure the deliverable does not narrate how the answer was constructed (Blocking: GLD-7).
- **Require Independent Review:** Have a second qualified reader independently derive contested records using only the disclosed set (Blocking: GLD-8).
- **Regenerate Entirely:** After any fixture extension, regenerate and diff the entire key, not just the newly added rows (Blocking: GLD-9).
- **Validate Passing Reasons:** Run `test.sh` on `solution/files/` and confirm every check passes for the right reasons and satisfies the written instructions (Blocking: GLD-10).

Special Attention

- **Mental Boundaries:** The boundary cases you resolved in your head but never explicitly wrote down are where keys fail most often.

- **Single Identity:** Golden files, installed files, and verifier expectations must share a single derivation identity (Blocking: GLD-11). Fixing a count in a CSV but failing to update `verifier.json` makes the task unsatisfiable; regenerate all from one source (VER-8).

Seventeen Patterns That Pass Your Golden and Fail a Correct Agent (7.14)

ID	Pattern	You Wrote	The Agent Wrote	Fix	Rule
O1	Numeric format fixed by string compare	1000	1000.00	Parse and compare numerically, or state the format	VER-6
O2	Float exact equality	0.30000000000000004	0.3	State precision; compare with a tolerance	VER-6
O3	Unstated rounding rule	2.5 to 2	2.5 to 3	State half-up or half-even	INS-5, DIS-3

O4	Date format	2024-01-15	15/01/2024	State the exact format with an example	INS-5, FIX-10
O5	Enum case sensitivity	none	None	State the literals verbatim in the instruction	INS-4
O6	Boolean or null representation	false	False, 0, "", null	State the exact token for each	INS-5
O7	Currency and unit decoration	1000	\$1,000.00	State whether symbols and separators are allowed	INS-5, VER-6

O8	Row order dependence	input order	sorted by id	Compare as a set, or state the sort order	VER-7
O9	Column order dependence	your order	alphabetical	Compare by column name, or state the order	INS-5, DIS-4
O10	JSON key order or value type	"count": 8	"count": "8"	State the JSON types; assert on parsed values	INS-5, VER-7
O11	Encoding and line endings	LF, no BOM	CRLF or a BOM	Normalise before comparing; state the encoding	VER-17

O1 2	CSV quoting style	a,b,c	"a","b","c"	Tolerate quotes in the template; never a rigid line match	VER-4, VER-17
O1 3	Whitespace and trailing newline	none	trailing newline	Strip before comparing	VER-17
O1 4	Prose graded by keyword	a word	synonym/other words	Grade figures inside the prose, never vocabulary	VER-3
O1 5	Unstated tie-break/precedence	your resolution	other defensible one	State the canonical rule, or accept both	INS-3

O1 6	Unstated boundary (greater vs at least)	your side	the other side	State inclusive or exclusive in words	INS- 2, FIX- 5
O1 7	Extra files or intermediates	none	scratch/skippe d file	Assert only on the named deliverable s	VER -16

7_8__Verifier Construction and Judge Checks

Verifier Construction and Judge Checks (Stages 7 & 8)

7. Verifier Design Principles

- **The Prime Directive:** An assertion is legitimate only if *every* output that satisfies the instruction passes it. A verifier should grade the task, not your specific solution. If a different, equally correct solution fails, the check is unfair.
- **Instruction-Driven Assertions:** Write every check looking exclusively at the instruction and governing documents, not your golden output. If a check requires an unstated format, add that sentence to the instruction.
- **Traceability (VER-2, VER-15, DIS-2):** Map every material requirement to a check and ensure every reward-bearing check cites the specific sentence it enforces. Boilerplate justifications are unacceptable.
- **Representation Contract (VER-1):** Define one consistent rule for case, whitespace, quoting, line endings, and record order, and apply it identically across every check.
- **Targeted Assertions (VER-12, VER-13, VER-14):** One check equals one fact. Check names must truthfully describe what they grade, and failures must report actionable `expected` and `actual` values.

Engine Capabilities and Parsing (VER-4)

- Use the engine's built-in parsers wherever possible (e.g., `json.read_file`, `csv.inspect_table`, `xlsx.read_cell`).
- **JSON:** Use JSONPath anchored to a single match (VER-19).
- **CSV Content:** Grade rows via regex on raw text using a tolerant template: anchored per line, order-free (`(?m)`), optional quotes, and flexible whitespace (VER-17).
- **Collection Extent (VER-5):** Always pair row-level presence checks with a total row/key count to prevent agents from hedging by dumping every possible answer.
- **Numbers:** Compare parsed numbers numerically (`approx_equals` or `equals`) with stated tolerances, never as strict strings (VER-6).
- **Negative Comparators (VER-18):** Always pair "must not contain" checks with an existence check to prevent empty files from passing.

- **Extraction Caps (VER-20):** Text extractions are capped at 90,000 characters; do not exceed this limit.

Grading Execution & Scoring Rules

- **Atomic Satisfaction (VER-8, VER-9, VER-26):** Derived summary checks must be mutually satisfiable with row checks. Grade self-reported summaries by recomputing them from the agent's *own* table, ensuring a wrong row decision is only penalized once.
- **Prose Grading (VER-3):** Grade prose only on extractable figures/identifiers or via sanctioned LLM checks, never on exact wording or formatting.
- **Path Validation (VER-16, VER-25, VER-28):** Read only named deliverables using `lstat` to reject symlinks, directories, or oversized files before grading.
- **Cosmetic Weights (VER-21):** Fold existence checks into content checks; do not let cosmetic checks inflate a proportional denominator.
- **Coverage & Dead Code (VER-22, VER-23):** Ensure every declared check executes and fails the specific decoy it was designed to catch.
- **Discrimination Harness (VER-24):** Ship a script verifying that wrong candidates fail for the correct reason.

8. Judge-Based Checks (LLM Rubrics) If a deterministic check cannot express a requirement, you may use an LLM rubric, but you must document why. Because any check flip can zero a strict-pass run, stability is paramount (LLM-13).

- **Prompt Design (LLM-2, LLM-3, LLM-5, LLM-6):** Prompts must ask one coarse, unconditional, binary yes/no question about substance. Do not grade layout or length, do not enumerate answer-key specifics, and explicitly state that extra content is tolerated.
- **Context Control (LLM-4, LLM-11, LLM-15):** The judge must read only the full, untruncated artifact it is grading, passed through its native source. Never pass the golden key to the judge.
- **Validation (LLM-7, LLM-8):** Probe rubrics heavily before shipping. They must pass all real outputs and fail naive/wrong outputs consistently across three repeats without flipping verdicts. Turing's team runs final stability repeats.
- **Environment Configuration (LLM-9, LLM-10, LLM-14):** Use a provider-valid model ID and override it with `$_JUDGE_MODEL` at runtime. If the network is `no-network`, open a route to the judge. Judge outages must trigger an errored check (CTRF `other`, exit code

2), never a failure.

9_10__Harness and environment setup

Stage 9. Build the harness and choose the reward shape

9.1 Choose and declare the shape (HAR-2).

Binary, proportional, or weighted with or without a gate: your choice, stated in `task.toml` metadata, in the README and in `review.csv`, and implemented exactly that way in `test.sh`. Whatever the shape, a cosmetic miss cannot erase correct work and one decision is charged once (HAR-3).

9.2 Steps

- **9.2.a** Write `reward.txt` on every path, including a crash (HAR-1); degrade cleanly on an empty workspace with a readable per-check message and a complete CTRF (HAR-8).
- **9.2.b** Read pass, fail, error, and weight totals from the engine's generated result payload, never from typed constants. (HAR-5).
- **9.2.c** Distinguish errored from failed: record an errored check as CTRF `other`; mark the run ineligible and exit 2. (HAR-6).
- **9.2.d** Install every dependency at build time; `test.sh` installs and downloads nothing (HAR-7).
- **9.2.e** Run the grader from its own directory with a controlled interpreter: absolute interpreter path, isolated mode, fixed `PATH`, `PYTHONPATH` unset, `--rootdir=/tests` and `-p no:cacheprovider` if pytest is the runner (HAR-9, EXP-8).
- **9.2.f** Guard and snapshot before grading (VER-25, EXP-7): validate each deliverable's path; copy valid deliverables into the fresh snapshot; grade only that snapshot.
- **9.2.g** Declare any harness-side check beside `verifier.json`, with a justification, so the executed set equals the declared set (VER-22).
- **9.2.h** Run the oracle locally, twice, from a clean checkout; both read exactly 1 with every check passing (HAR-10). Run the empty submission (below 1) and the symlink substitution (exactly 0) (HAR-11).
- **9.2.i** Measure and record the reward floor using every task-applicable degenerate submission: a majority- or default-class answer; a naive solver that ignores the hardest source; keyword-only prose and an instruction-copy when a prose deliverable exists; and a trivial transform of an input (HAR-4)

Checklist for Stage 9

- **HAR-1** (Blocking) `reward.txt` is written on every path. Client subcheck: `verifier_execution`.
- **HAR-2** (Blocking) The reward shape is declared and the code matches. Client subcheck: `aggregation_normalization`.
- **HAR-3** (Blocking) Whatever the shape, a cosmetic miss cannot erase correct work and

one decision is charged once. Client subcheck: [scoring_proportionality](#).

- **HAR-4** (Blocking) The reward floor is measured, not inferred. Client subcheck: [rollout_legitimacy](#).
- **HAR-5** (Blocking) No hard-coded fallback constants. Client subcheck: [aggregation_normalization](#).
- **HAR-6** (Blocking) Errored checks are distinguished from failed checks. Client subcheck: [judge_failure_handling](#).
- **HAR-7** (Warning) Verifier dependencies are pinned and installed at build time. Client subcheck: [manifest_environment_consistency](#).
- **HAR-8** (Blocking) The harness degrades cleanly on no agent output. Client subcheck: [verifier_execution](#).
- **HAR-9** (Blocking) The verifier runs from its own directory with an explicit rootdir. Client subcheck: [state_spoofing](#).
- **HAR-10** (Blocking) The oracle reproduces exactly 1.0 locally with the shipped [test.sh](#) before submission. Client subcheck: [e2b_strict_pass](#), [modal_strict_pass](#).
- **HAR-11** (Blocking) An empty submission scores below 1.0, and the symlink-substitution run scores 0. Client subcheck: [state_spoofing](#).

Stage 10. Set the environment and the manifest10.1 Dockerfile steps.

- **10.1.a** Copy `input/` only; never `solution/`, `tests/`, `README` or `.` (ENV-1, DIS-7).
- **10.1.b** Pin the base image by digest; the build is deterministic (ENV-3).
- **10.1.c** Mark the inputs read-only, but treat this as advisory because the agent runs as root. No reward-bearing content check may read `/app/input`; all expected values must come from the pinned inputs. After the agent phase, an evidence-only integrity step computes each input's SHA-256, compares it with the pinned hash as evidences (ENV-2, ENV-5, FIX-11).
- **10.1.d** Hygiene: run `apt-get update`, install packages, and remove package lists in one layer; install apt packages from a dated snapshot repository or pin them to exact versions; install Python dependencies from a complete exact-version lock; use a `.dockerignore`; use no heredocs for source files; apply `chmod -R` only to the input tree; and use LF endings (ENV-3, ENV-8).
- **10.1.e** Create the parent directory of every declared artefact (ENV-10).
- **10.1.f** Size timeouts from measured runs with headroom; never lower them to manufacture failures (ENV-9).
- **10.1.g** Verify containment in a fresh container: a whole-filesystem search for `solve.sh`, `test.sh`, `verifier.json`, the golden files and the generator prints nothing; `/tests` and `/logs/verifier` are absent in the agent phase (ENV-7, EXP-6).

10.2 task.toml steps.

- **10.2.a** Loads under the client's Harbor version; check with the loader, not by eye (TOML-1).
- **10.2.b** `[task].name = "obi/<folder-name>"`, unique; keywords include `non-connector` and `offline` (TOML-2, TOML-3).
- **10.2.c** `artifacts` at top level, absolute `/app` paths, one per deliverable, spelled identically everywhere (TOML-4, PKG-10).
- **10.2.d** `[metadata.verifier_judge].model = "${JUDGE_MODEL}"`; `[verifier.env]` holds `${VAR}` references only (TOML-5, TOML-6).
- **10.2.e** `[environment]: os = "linux"`, `mcp_servers = []`, a measured `build_timeout_sec`, no `docker_image`, `network_mode` per ENV-4, never both `allow_internet` and `network_mode` (TOML-8).
- **10.2.f** `[verifier].collect = []`, `environment_mode` unset (TOML-9).
- **10.2.g** No personal information or credential value; no hidden-rule markup in metadata; `source_config` points at the mined source; a one-line description (TOML-7, TOML-10, TOML-12, TOML-13).
- **10.2.h** Every number restated elsewhere agrees (TOML-11, PKG-12).

10.3 Network mode (ENV-4)

Harbor's enum is `no-network`, `public`, or `allowlist`; there is no `none`. An offline task without a judge-based check uses `network_mode = "no-network"`. A task with a

judge-based check follows 8.2.k: until environment `no-network` plus verifier `allowlist` has been validated on both E2B and Modal, use `public`; once validated, use that split configuration.

Checklist for Stage 10

- **ENV-1** (Blocking) The Dockerfile copies `input/` only. Client subcheck: `golden_isolation`.
 - **ENV-2 (Blocking)** Inputs are marked read-only; no reward-bearing content check reads live inputs. Client subcheck: `state_spoofing`.
 - **ENV-3** (Blocking) The base image is pinned by digest and the build is deterministic. Client subcheck: `manifest_environment_consistency`.
 - **ENV-4 (Blocking)** `no-network` for offline tasks without judge-based checks, with the documented temporary routing exception for tasks that contain judge-based checks. Client subcheck: `golden_isolation`, `golden_or_internal_access`.
 - **ENV-5 (Blocking)** Post-run input hashes are compared with the pinned hashes and the result is recorded; any mismatch is reported as an integrity failure. Client subcheck: `state_spoofing`.
 - **ENV-6** (Warning) Stale artifacts from a prior run do not produce a pass. Client subcheck: `state_spoofing`.
 - **ENV-7** (Blocking) Containment verified in a fresh container. Client subcheck: `golden_isolation`.
 - **ENV-8** (Warning) Dockerfile hygiene. Client subcheck: `manifest_environment_consistency`.
 - **ENV-9** (Warning) Timeouts are sized from measured runs with headroom. Client subcheck: `harness_runtime_compatibility`.
 - **ENV-10** (Blocking) The parent directory of every declared artifact exists in the image. Client subcheck: `artifact_materialization_export`.
-
- **TOML-1** (Blocking) Loads under the client's Harbor version. Client subcheck: `cross_artifact_consistency`.
 - **TOML-2** (Blocking) `[task].name = "obi/<folder-name>"`, unique. Client subcheck: `cross_artifact_consistency`.
 - **TOML-3** (Blocking) `keywords` include `"non-connector"` and `"offline"`. Client subcheck: `cross_artifact_consistency`.
 - **TOML-4** (Blocking) `artifacts` at top level, absolute `/app` paths, one per deliverable, names identical everywhere. Client subcheck: `artifact_contract_consistency`.
 - **TOML-5** (Blocking) No literal judge model ID; `[metadata.verifier_judge].model = "${JUDGE_MODEL}"`. Client subcheck: `judge_execution_validity`.
 - **TOML-6** (Blocking) `[verifier].env` uses `${VAR}` references only. Client subcheck: `manifest_environment_consistency`.
 - **TOML-7** (Warning) No `metadata.scenario_description` with hidden-rule markup. Client subcheck: `workflow_realism`.
 - **TOML-8** (Blocking) `[environment]` fields. Client subcheck: `manifest_environment_consistency`.

- **TOML-9** (Blocking) `[verifier].collect = []`; `environment_mode` unset. Client subcheck: `manifest_environment_consistency`.
- **TOML-10** (Blocking) No personal information or credential values. Client subcheck: `golden_isolation`.
- **TOML-11** (Blocking) Every number restated elsewhere agrees. Client subcheck: `cross_artifact_consistency`.
- **TOML-12** (Warning) `metadata.source_config` points at the mined source. Client subcheck: `cross_artifact_consistency`.
- **TOML-13** (Warning) `[task].description` is one line and describes the task. Client subcheck: `workflow_realism`.

11_12__Run the Task

Proving the Task, Running it & Security Sweeps (Stages 11 & 12)

11. Prove the Task Before Any Trial You must find verifier defects locally before executing agent runs. Ship a script that performs these checks and record its results (VER-24).

11.1 The Equivalence Suite: Must Still Pass (VER-11) Copy the golden output and perturb it in ways that preserve meaning; the verifier must still pass it.

ID	Perturbation	Catches	If it fails
E1	Shuffle data row order	O8	Blocking
E2	Reorder columns, keeping headers, where the instruction leaves order open	O9	Warning
E3	Reorder JSON keys	O10	Blocking
E4	Quote every CSV field	O12	Blocking
E5	Convert LF to CRLF	O11	Warning

E6	Add or remove the trailing newline	O13	Blocking
E7	Prepend a UTF-8 BOM	O11	Warning
E8	Reformat integers as N.0 and back, where the format is open	O1	Blocking
E9	Add and strip trailing decimal zeros, where the format is open	O1	Blocking
E10	Pad fields with surrounding spaces	O13	Warning
E11	Change JSON numbers to strings, where types are open	O10	Warning
E12	Paraphrase every sentence of a prose deliverable, keeping all figures	O14	Blocking

E13	Delete an unrequired intermediate file	O17	Blocking
E14	Add an unrelated scratch file to <code>/app</code>	O17	Blocking

11.2 The Breaking Suite: Must Now Fail (VER-11, VER-5, VER-8, FIX-5) Break the golden output for real; it must fail on the specific check written for that break. Run B5 first, as boundary rows are the most common source of unwritten rules and underspecified instructions.

ID	Perturbation	A pass here means
B1	Flip one classification or answer value	that row's correctness is ungraded
B2	Delete one data row	completeness is ungraded
B3	Append one spurious row	you check presence, not the whole collection

B4	Change one summary count by one	derived figures are ungraded
B5	Flip the boundary-case row to the other side	the threshold rule is ungraded
B6	Empty every deliverable file	only existence is checked
B7	Delete one deliverable	that deliverable is ungraded
B8	Replace all output with <code>{ }</code> or a bare header	your verifier does not evaluate

11.3 - 11.7 Diagnostic Probes & State-Spoofing

- **The Decoys (VER-23):** Build deliverables based on the expected wrong readings from Stage 3; each must fail *only* on the rows it changes.
- **The Degenerate Solvers:** Record the reward for the majority-class solver, the naive solver, a keyword memo, the instruction pasted as a memo, and trivial input transforms (HAR-4, EXP-1, EXP-2, EXP-4, EXP-10, EXP-11).
- **State-Spoofing Runs (EXP-3, EXP-5, EXP-6, EXP-7, EXP-8, HAR-11):** Run symbolic links to reference files (must score 0). Reject hard links, directories, and oversized files.

Plant `conftest.py` and `sitecustomize.py` with shadow `python3` paths; ensure they have no effect. Run an empty workspace; it must score below 1.

- **Containment & Read-Only Claim (ENV-7, ENV-2):** Search the fresh container to ensure it prints nothing. Confirm that `echo x >> /app/input/<file>` succeeds as root, and that nothing under `tests/` reads `/app/input`.
- **Local Stability (EVD-11):** If the task has a judge-based check, optionally run the verifier three times to ensure per-check vectors are identical.

11.8 The Security Sweep (SEC-1 to SEC-6) Perform one pass over the package to catch credential values, undeclared network calls, obfuscated payloads, prompt injection, and destructive operations.

11.9 Special Attention

- **Negative Comparators:** An empty CSV passes every "must not contain" check. Pair every negative check with a positive one on the same file (VER-18).
-
- **Tightening/Broadening:** If you tighten a check, re-run the equivalence suite to ensure valid variants aren't rejected. If you broaden coverage, re-run the decoys to ensure the harness still catches them.
-

12. Run the Task and Read the Runs

12.2 Classify Every Non-Passing Run (TRL-1)

Before counting a run, classify it strictly in this order:

1. `exception_info` set, or setup failed: **Infrastructure** (Exclude and re-run).
2. Ran to timeout while work continued: **Sizing** (Raise the timeout per ENV-9; exclude and re-run).
3. Wedged (stale observations, no echo): **Infrastructure** (Exclude and re-run).
4. Stop reason `length` before deliverables: **Harness limit** (Check input size per FIX-16; exclude and re-run).
5. Completed, no deliverables, sound attempt: **Valid failure**.
6. Deliverables present with reward below 1: Proceed to normalisation comparison.

Note: Only valid failures count toward the pass band (TRL-5). Use client phase names (TRL-6). Runs with exceptions, missing rewards, invalid trajectories, or zero actions are ineligible (TRL-7).

12.3 & 12.4 Normalising and Comparing Runs (TRL-2, TRL-3, TRL-4)

- Parse the agent's and golden outputs, sort rows/keys, strip whitespace, coerce numbers, and casefold unfixed fields. If they are equal but the agent scored < 1 , you have a verifier defect (overfit).
- If they differ on a field the disclosed set determines, the failure counts.
- If they differ on an open field, the instruction is underspecified.
- If all four runs agree with each other but differ from the golden key, your key is likely wrong; re-derive it manually.

12.5 - 12.8 Failure Analysis (DIF-4, DIF-8, TRL-8)

- If all four runs fail on a formatting check, the task is overfit, not hard.
- A 0/4 score means the task is either genuinely hard or the verifier is rejecting correct work. Stronger models may be used diagnostically, but never reported (DIF-8).
- Annotate each failed run against the expected wrong readings declared in Stage 3.
- Group failures by check and cause, assigning one uniform classification per group (TRL-8).

13_14__Evidance and Package

Assemble the Evidence (Stage 13)

Layout and Contents

- **Directory Layout (EVD-1):** Ship exactly the `evaluations/solvability/r1/` and `evaluations/difficulty/r1/` to `r4/` folders. Turing's team will add `oracle/rN/` and `stability/rN/`—do not ship these yourself, and leave no loose files.
- **Run Folder Completeness (EVD-2, EVD-13, EVD-14, EVD-15):** Each run folder must contain the unedited `agent/trajectory.json`, `result.json`, `config.json` (with redacted values), `verifier/reward.json`, `verifier/reward.txt`, `verifier/ctrf.json`, `verifier/test-stdout.txt`, `artifacts/manifest.json`, and the exact deliverables graded inside `artifacts/app/`. Difficulty runs must also include `verifier/verifier_summary.json`.

Trial Identity and Replay Validation

- **Consistent Identity (EVD-3, EVD-4, EVD-6, EVD-7):** Every file in a run folder must agree on `trial_name` and `task_checksum`, with the checksum matching the submitted package. Difficulty runs must consist of exactly four consistent, non-oracle GLM-5.2 runs. The solvability run must be a complete, non-oracle run scoring exactly 1.0 by any model.
- **Golden Trajectory (PKG-8):** `solution/golden_trajectory.json` must be a reward-1.0 model run, distinct from the oracle.
- **Replay Verification (EVD-8, EVD-9):** Re-running the shipped `test.sh` on `artifacts/app/` must yield the exact recorded reward. Trial outputs must strictly use vocabulary defined by the instruction, and recorded check names must exist in the shipped verifier.

Exclusions and Regeneration

- **Run Exclusions (EVD-10):** Remove, replace, and never count infrastructure, harness-limit, and sizing runs.
- **Regeneration Rule (EVD-12, PKG-11):** Any edit to an input, instruction, verifier, solution file, or harness changes the checksum. This instantly voids every trial, trajectory, and QC report, requiring full regeneration. (Optional: Local stability checks per EVD-11).

Write the Package Files and Submit (Stage 14)

Structure and Consistency

- **Clean Package (PKG-1, PKG-9, PKG-10):** The submission must perfectly match the required folder structure. Remove all scratch, backup, cache, or duplicate files. Deliverable names must be spelled identically across the instruction, verifier sources, `solution/files/`, and the `artifacts` list.

README Requirements (PKG-2, PKG-3, PKG-4, PKG-5, PKG-12) Write the README for a human reviewer. Every figure must be read directly from its supporting file, and quantities must never be stated with two conflicting values. Only shipped evidence under `evaluations/` may be quoted with numbers. Follow this strict section order:

1. **Task Description:** A paragraph explaining the task, the population, and why the population is not handed over.
2. **Departures:** A file-by-file log of changes from the mined version and why they were made (PKG-2).
3. **Declarations:** The reward shape (matching `task.toml` and `test.sh`), declared checks by kind, network mode, and judge model (with reasoning) (PKG-5).
4. **Golden Key Derivation:** Results of the manual Stage 6 key checks.
5. **Difficulty Design:** The expected wrong readings and the specific rows they change, declared prior to trials (DIF-4).
6. **Probes & Solvers:** Results from the equivalence suite, breaking suite, decoys, and degenerate submissions (with rewards) (VER-11, VER-23, HAR-4).
7. **Environment Tests:** Confirmation of containment search, oracle runs, empty/symlink runs, and planted-file runs (ENV-7, HAR-10, HAR-11).
8. **Trial Results:** Each run's reward, classification, and matched wrong reading.
9. **Isolation Proof:** Explicit confirmation that `solution/` or `tests/` are absent from the agent's container (PKG-4).
10. **Reproduction Commands:** Instructions for the reviewer to reproduce the probes and environment tests.

`review.csv` Construction (PKG-6)

- **Column Order:** `review_check`, `status`, `review_notes`, `change_made`, `what_to_record`.

- **Row Order:** Layer 1 (Package consistency, Clarity and scope, Realism and leakage); Layer 2 (Difficulty, Solvability, Stability); Layer 3 (Oracle mode); Layer 4 (Environment and files, Connectors, MCPs, and CLIs, Deliverables and artifact quality); Layer 5 (Verifier coverage and fairness, LLM judge consistency, Reward hacking and exploitability); Cross-trial (Calibration).
- **Honest Statuses:** Use **PASS** only if no changes were made. Use **FIXED_AND_VERIFIED** to record a change and re-measurement. Use **N/A** only for the connector row, and for the judge-consistency row when the task has no judge-based check.
- **Evidence:** Every note must cite a specific file, run, or check; a measurement claim without a measurement is a defect. Provide the required one-line summary in [what_to_record](#).

Additional Tooling Checks (PKG-7, PKG-13, PKG-14)

- **QC Report:** The [qc_report.html](#) must reflect the tool's output at the submitted checksum.
- **Realism:** Deliverable carriers must be realistic for the domain, with the choice justified.
- **Template Cleanup:** All original mined template defects must be removed.

15__Checklist

Appendix A.15 PKG: Package Artefacts

Structure, Delivery, & Cleanup

- **Strict Folder Structure (PKG-1):** The zipped folder must strictly match the client's expected structure, containing exactly: `task.toml`, `instruction.md`, `review.csv`, `qc_report.html`, `README.md`, `tests/`, `environment/`, `solution/`, and `evaluations/`. No extraneous files or caches (e.g., `.pytest_cache`, `__pycache__`, `.git`) are permitted at the top level.
- **Extraneous Files (PKG-9):** Ensure absolutely no scratch files, backups (`.bak`, `~`), editor swap files, caches, `.DS_Store` files, or unreferenced data exist anywhere in the package.
- **Deliverable Realism (PKG-13):** Deliverable formats (e.g., workbooks, CSV/JSON summaries) must be realistic for the domain, and this choice must be explicitly justified in the README.
- **Output Inventory Consistency (PKG-10):** Deliverable names must be spelled identically across `instruction.md`, `verifier.json`, `solution/files/`, and the `artifacts` list in `task.toml`.

README Documentation Requirements

- **Log Departures (PKG-2):** The README must record every change made from the initial mined version on a file-by-file basis, detailing what the file was, what it is now, and why it was changed.
- **Factual Accuracy (PKG-3):** Every figure in the README must match the evidence files perfectly. Read values directly from files, never quote from memory, and only describe shipped evidence.
- **Reviewer-Facing Context (PKG-4):** The README must be written for the reviewer and explicitly state what the agent cannot see, such as confirming that `solution/` and `tests/` are verified as absent from the agent's container.
- **Required Declarations (PKG-5):** The README must explicitly declare the reward shape, network mode, judge model, and the expected wrong readings (listed before running the battery).
- **No Conflicting Numbers (PKG-12):** Stating two different bare numbers for a single quantity anywhere in the package is a defect (even for historical notes); describe history

qualitatively instead.

Validation, Evidence, & Propagation

- **Honest Review Tracking (PKG-6):** `review.csv` must be rigorously accurate, utilizing specific statuses (`PASS` if unchanged, `FIXED_AND_VERIFIED` with recorded changes, or `N/A` for missing judge/connector elements). Every note must cite specific files, runs, or checks, and measurement claims must include actual metrics.
- **Automated QC Reports (PKG-7):** The `qc_report.html` must be the unedited output generated by the QC tool at the submitted checksum.
- **Valid Golden Trajectories (PKG-8):** `golden_trajectory.json` must be a real agent run scoring exactly 1.0 based solely on the instructions. It must serve as independent evidence (distinct from the `solvability/r1` oracle run) and contain no references to golden files or verifier logic.
- **Strict Change Propagation (PKG-11):** Any edit to the instruction, inputs, verifier, golden files, harness, or Dockerfile changes the checksum. You must immediately regenerate the trials, trajectory, and QC report to ensure provenance validity.

Mined Template Adjustments

- **Remove Template Defects (PKG-14):** All inherent defects from the mining pipeline's template must be fixed and logged. This includes removing ungradeable instructions (e.g., "must be your final action"), harness meta-lines, empty `artifacts` lists, unpinned base images, boilerplate justifications, existence-only checks weighted improperly, dead test variables, and overly simplistic fixtures.

Open Questions

Open Questions

Unresolved ambiguities, policy questions, and status rulings for team and client discussions.

Status Legend: ``resolved`` (Decision finalized) · ``team`` (Needs Turing ruling) · ``client`` (Needs client confirmation) · ``open`` (Unresolved)

Table of Contents

- 1. **1. Evaluation & Pass Criteria**
 - 1.1. Q1: Difficulty band
 - 1.2. Q2: LLM-judged checks vs exact-1.0 requirements
 - 1.3. Q3: Binary reward + LLM check
 - 1.4. Q7: Client's six-trial matrix
- 2. **2. Verification, Tooling & Environment**
 - 2.1. Q6: solve.sh installs precomputed files
 - 2.2. Q10: Hard-coded judge model ID in the verifier
 - 2.3. Q12: no-network environments and an LLM-judged verifier
 - 2.4. Q13: The 90,000-character extraction cap
- 3. **3. Evidence, Artifacts & Structure**
 - 3.1. Q4: provenance.json per evaluation run
 - 3.2. Q5: Stability evidence shape
 - 3.3. Q8: Optional oracle/ and stability/ under evaluations/
 - 3.4. Q9: LLM check present ⇒ stability evidence must ship
- 4. **4. Specification & Prompting**
 - 4.1. Q11: Population count in the instruction

1. Evaluation & Pass Criteria

Q1 · Difficulty band

Status: **RESOLVED** (Turing ruling 2026-09-04: at most 3 of 4 strict passes is acceptable; 4/4 is rejected. The client's shipped script still fails at more than 2 passes; the team aligns it with the client.)

Conflict	Turing target is ≤ 3 of 4 GLM-5.2 strict passes. The client's shipped <code>`audit_evaluations.py`</code> fails a task at <code>`passes > 2`</code> ($\leq 2/4$), and
----------	---

	<code>`task-review-rubric.toml`</code> (<code>`layer2_difficulty__strict_pass_distribution`</code>) says "no more than two earn reward exactly 1.0".
Assumed & Risk	Assumed for now: $\leq 3/4$, per Turing. Risk: any task with exactly three passes is rejected by the client gate as it stands. Needs: client confirmation that the script and rubric will move to $\leq 3/4$, or Turing reverting to $\leq 2/4$.

Q2 · LLM-judged checks vs exact-1.0 requirements

Status: RESOLVED (Turing ruling 2026-09-04: judge-based checks are allowed with no limit on their number; every check must meet LLM-2 to LLM-15; the flip arithmetic in LLM-1 stands as the reason to keep each one justified and probed.)

Conflict	The oracle must score exactly 1.0 in both E2B and Modal; a solvability pass must be exactly 1.0; the client's stability gate requires identical per-check result vectors across three repeats on frozen deliverables. An LLM-judged check that flips once on identical input fails <code>`layer2_stability__per_check_stability`</code> and <code>`reward_equality`</code> . Also, the client's oracle judge runs on a different endpoint (Fireworks <code>`glm-5p2`</code>) from the vendor proxy (<code>`wandb-glm/glm-5.2`</code>), so a rubric probed stable on one may not be stable on the other.
Assumed / Reframing	Assumed for now: LLM checks permitted with limits (≤ 1 per task; probed on all real memos plus near-misses, 3 repeats, no flip). Reframing (2026-09-04): under strict pass (<code>`reward == 1.0`</code>) the "cannot alone zero a run" limit protects nothing — one flip of one check turns 1.0 into a fail. The property that matters is <i>must not flip</i> , evidenced by repeats. With n LLM checks each flipping with probability p on identical input, a correct submission reaches strict 1.0 with probability $(1-p)^n$: at $p=0.05$, one check $\rightarrow 95\%$, three $\rightarrow 86\%$, ten $\rightarrow 60\%$.
Options & Engine Update	Options tabled: (A) no LLM checks — memo graded on extracted figures or ungraded; (B) at most one, coarse binary, unconditional phrasing, judge sees only the artifact, probed 3 $\times$ on the Fireworks tuple, 3 stability repeats shipped; (C) advisory zero-weight checks; (D) majority-of- N via repeated entries in <code>`config.models`</code> . Draft recommends A by default, B by exception. Update (2026-09-04): option D is not available under client execution. <code>`runner.load_spec`</code> replaces <code>`config.models`</code> with the single <code>`JUDGE_MODEL`</code> value whenever set. Options reduce to A, B, C. Needs team ruling among A–C and probe responsibility for B.

Q3 · Binary reward + LLM check

Status: TEAM (Needs Turing ruling)

Ruling & Consequence	Ruling taken: a binary-shaped task may carry an LLM check, and the "cannot alone zero a run" limit is dropped for it. Consequence recorded: a model's judgement can then zero a correct run. This is the task-1 rejection mechanism with a model instead of a regex. Worth revisiting once Q2 is settled.
---------------------------------	---

3. Evidence, Artifacts & Structure

Q4 · provenance.json per evaluation run

Status: RESOLVED (Turing ruling 2026-09-04: not required; EVD-5 withdrawn. Trial identity rests on result.json, config.json, lock.json and the trajectory.)

Gap & Assumed	Gap: Client README and <code>run_glm_task_qc.py</code> expect <code>evaluations/difficulty/rN/provenance.json</code> with <code>axis</code> , <code>actor_model</code> , <code>source_trial</code> , <code>task_digest</code> . The deterministic script does not check it; the GLM reviewer and human procedure do. Neither existing package ships one. Assumed for now: required, four fields, <code>task_digest</code> must equal the submitted package's content hash and each <code>result.json</code> 's <code>task_checksum</code> . Needs client confirmation.
--------------------------	--

Q5 · Stability evidence shape

Status: TEAM (Needs Turing ruling)

Gap & Needs	Gap: The client stability gate needs <code>result.json</code> fields none of our packages carry: <code>frozen_trajectory_sha256</code> , <code>checks</code> , <code>regraded_checks</code> ; plus byte-identical <code>config.json</code> and <code>lock.json</code> across ≥ 3 repeats. Turing's team produces stability evidence; trainers do a lightweight local check only. Needs: confirmation that Turing's tooling emits these fields, and a decision on whether <code>stability/</code> may be shipped by trainers at all (see Q8).
------------------------	---

2. Verification, Tooling & Environment

Q6 · solve.sh installs precomputed files

Status: RESOLVED (Turing ruling)

Ruling &	Ruling taken: permitted, no derivation required. Consequence recorded:
---------------------	--

Consequence	the oracle run proves only that a file matches itself (the client's own guide says this). Key-correctness rules (self-consistency across identical inputs, every rule applied to every record, boundary rows re-derived by hand) are therefore mandatory as the only controls on the key.
--------------------	---

Q7 · Client's six-trial matrix

Status: TEAM (Needs Turing ruling)

Observation & Needs	Observation: The client's Layer 4 runs three lanes × two attempts — GLM-5.2/terminus-2, Fable-5/claude-code, GPT-5.6/codex — at maximum reasoning effort, and flags ≥5 of 6 strict passes as too easy (<code>`layer5_cross_trial_calibration__calibration_profile`</code>). A task that stumps GLM at 1–2/4 but is solved 2/2 by both stronger models lands at 5–6/6 and is flagged. Trainers cannot run those lanes. Needs: whether the Turing standard should require difficulty that survives stronger models, and what evidence a trainer could offer for it.
--------------------------------	---

Q8 · Optional oracle/ and stability/ under evaluations/

Status: RESOLVED (Turing ruling 2026-09-04: oracle/ and stability/ are produced and added by Turing's team, never by the trainer; a trainer may run a local stability check but ships nothing from it. EVD-1, EVD-11 and LLM-8 rewritten.)

Ruling & Gap	Ruling taken (Turing): <code>`solvability/`</code> and <code>`difficulty/`</code> required; <code>`oracle/`</code> and <code>`stability/`</code> optional, same <code>`rN`</code> layout. Gap: the client structure says "named folders only, nothing loose" and lists only the two required ones; <code>`audit_evaluations.py`</code> does read a <code>`stability/`</code> axis. Needs: client confirmation that <code>`oracle/`</code> is tolerated and <code>`stability/`</code> from a vendor is accepted rather than ignored.
-------------------------	--

Q9 · LLM check present ⇒ stability evidence must ship

Status: RESOLVED (by Q8 ruling, 2026-09-04: Turing's team runs and ships the three repeats; local check optional.)

Fact & Conflict	Fact: <code>`audit_collection.py`</code> raises an automatic warning whenever <code>`llm_rubrics > 0`</code> and fewer than three stability repeats are found in the package, and a blocker when repeat rewards disagree. So a task with any LLM check must ship client-shaped stability evidence at delivery. Conflict: Turing's team produces stability evidence; trainers do a lightweight local
----------------------------	--

	check only (Q5). Needs: who produces the three shipped repeats before delivery for LLM-checked tasks, and whether that makes option A in Q2 the practical default.
--	---

Q10 · Hard-coded judge model ID in the verifier

Status: CLIENT (Needs client confirmation)

Fact & Needs	Fact: <code>`audit_collection.py`</code> flags the literal <code>`"openai/glm-5.2"`</code> in <code>`tests/manifest.json`</code> as a blocker ("not deployed on the configured Fireworks OpenAI-compatible endpoint"). Our verifier engine carries the equivalent in <code>`tests/verifier.json`</code> → <code>`config.models`</code> ; the h125 package has exactly <code>`["openai/glm-5.2"]`</code> there. The runtime <code>`JUDGE_MODEL`</code> override makes it work, but the static string is a flag risk. Assumed for now: the standard forbids a hard-coded judge model ID; <code>`config.models`</code> must hold a provider-valid ID and the README must say <code>`JUDGE_MODEL`</code> governs. Needs: client confirmation whether the check extends to <code>`verifier.json`</code> , and which static value they accept.
-------------------------	---

4. Specification & Prompting

Q11 · Population count in the instruction

Status: TEAM (Needs Turing ruling)

Observation & Needs	Observation: An instruction that states the count of the population the deliverable must derive ("the 40 tracks", "one row per product for all 142 preparations") leaks part of the answer. h125's README: "the population is not handed over." Assumed for now: the instruction may name the input files and the unit of audit, never the count of units. Needs: confirmation this is acceptable to the client's clarity reviewers, who may read an unstated count as under-specification.
--------------------------------	--

Q12 · no-network environments and an LLM-judged verifier

Status: TEAM / CLIENT (Needs ruling & confirmation)

Ruling & Interaction	Ruling taken: offline tasks declare <code>`[environment] network_mode = "no-network"`</code> (Harbor's enum is <code>`no-network`</code> / <code>`public`</code> / <code>`allowlist`</code> ; there is
---------------------------------	---

	no <code>"none"</code>). Interaction: an LLM-judged check needs a route to the judge at verify time. Harbor exposes per-phase overrides (<code>[verifier].network_mode</code> , <code>[verifier].allowed_hosts</code>), so the intended shape is environment <code>no-network</code> + verifier <code>allowlist</code> naming the judge host. Needs: one oracle run on each provider (E2B, Modal) with that configuration before the rule ships; until then, a task with an LLM check keeps <code>public</code> .
--	--

Q13 · The 90,000-character extraction cap

Status: TEAM (Needs Turing ruling)

Fact & Needs	Fact: Every <code>extract_text</code> source is capped at <code>max_content_chars</code> = 90,000, a <code>run_verifier</code> parameter no <code>verifier.json</code> can change and neither <code>test.sh</code> shape sets. A deliverable over the cap is silently truncated before any regex runs; rows beyond it fail their checks with no diagnostic. Assumed for now: the standard caps text-graded deliverables at a stated size well below 90k and routes large tables to JSON summaries or <code>inspect_table</code> counts. Needs: a number (proposed 60,000 characters per text-graded file), and whether Turing's <code>test.sh</code> should raise the cap explicitly.
-------------------------	--

Add new items below with the next number. When an item is resolved, change its status and record the ruling and date rather than deleting it.